BidForge AI

AI-Powered Bid & Proposal Response Engine

Project Report

CUST Hackathon 2026 — Problem #1 (TEKROWE)

Industry Vertical: Procurement, Sourcing & Contract Management

Difficulty Level: Advanced

Date: June 12, 2026

Table of Contents

1. Executive Summary

BidForge AI is an end-to-end, AI-powered Bid & Proposal Response Engine that automates the most time-intensive stages of responding to RFPs, RFQs, and Tenders. Bid teams typically spend 60-80% of their time reading lengthy tender documents, extracting compliance requirements, cross-referencing internal capability libraries, and drafting narrative responses. BidForge AI compresses this multi-hour manual workflow into minutes.

The system ingests an RFP document (PDF or DOCX), extracts mandatory requirements, evaluation criteria, deadlines, and budgets using a hybrid LLM + NER pipeline, matches every requirement against a 50-record Company Capability Library using hybrid Retrieval-Augmented Generation (RAG), auto-drafts a structured proposal with live streaming, flags compliance gaps, and produces an explainable win-probability score with a GO / CONDITIONAL / NO-GO recommendation. The final proposal — including an executive summary, technical narrative, compliance matrix, and score summary — exports to a professionally formatted DOCX document.

- **End-to-end automation:** upload → parse → extract → match → draft → score → export, with a separate workspace per bid.
- **All four required AI components:** LLM (Llama 3.3 70B), hybrid RAG (FAISS + structured re-ranking), deterministic NER, and a trained win-probability model.
- **Measured impact:** pipeline timing instrumentation demonstrates >90% reduction in bid preparation effort versus a 6-hour manual baseline — well beyond the 50% target.
- **Human-in-the-loop:** bid managers review, edit, and approve every AI-generated section before export.

2. Problem Statement & Background

Organizations in the public and private sector issue thousands of RFPs, RFQs, and Tenders every year. A typical enterprise bid team handles 40-120 bids annually, with individual bid documents ranging from 50 to 500+ pages. The response process is highly repetitive, error-prone, and time-intensive: missing a single mandatory compliance clause or submitting a poorly structured technical narrative can result in outright disqualification, directly impacting revenue.

The hackathon challenge (Problem #1, TEKROWE) required a system that can:

- Ingest an RFP/RFQ/Tender document (PDF or DOCX) and automatically extract mandatory requirements, evaluation criteria, submission deadlines, and question/answer sections.
- Match extracted requirements against a pre-loaded Company Capability Library (past project summaries, certifications, case studies).
- Auto-draft a structured proposal response with relevant content mapped to each question/section.
- Flag compliance gaps where the organization lacks evidence or capability.
- Score the bid opportunity using win-probability heuristics and make a GO/NO-GO decision.

- Demonstrate at least a 50% reduction in manual bid preparation effort.

3. Solution Overview

BidForge AI is organized around the concept of a workspace — an isolated container created for each RFP that tracks the bid through a six-stage lifecycle:

Stage	What Happens	AI Involved
1. Upload	RFP (PDF/DOCX) uploaded; dedicated workspace created	—
2. Parse & Extract	Text extracted page-by-page; requirements, deadlines, budgets, criteria identified	LLM + NER cross-check
3. Match	Each requirement matched against the Capability Library with pass / partial / fail verdicts	Hybrid RAG + LLM judge
4. Draft	Proposal sections generated per requirement, streamed live to the editor	LLM (temp 0.7)
5. Score	Win probability computed; GO / CONDITIONAL / NO-GO decision issued	ML model + heuristics
6. Export	Professional DOCX assembled: cover, executive summary, narrative, compliance matrix, score	LLM exec summary

At every stage the bid manager remains in control: requirements can be annotated and marked done, compliance verdicts carry confidence scores and gap notes, drafted sections are editable and must be approved, and competitor presence can be set manually to refine the score.

4. System Architecture

The system follows a clean three-tier architecture: a React single-page application, a FastAPI asynchronous backend, and a Supabase (PostgreSQL) persistence layer, with in-process AI services for embedding, retrieval, and model inference.

4.1 Backend

- **FastAPI + Uvicorn:** fully asynchronous request handling; long-running drafting is delivered over Server-Sent Events (SSE) so the UI renders tokens as they are generated.
- **Document parsing:** pypdf for PDFs (with page markers for traceability) and python-docx for DOCX, including table-cell extraction; scanned/image-only documents are rejected with a clear error.
- **Concurrency control:** up to three LLM chunk/batch calls in flight simultaneously, with exponential backoff (2s/4s/8s) on rate limits.
- **LLM resilience:** GROQ is the primary provider with automatic fallback to OpenRouter, and a file-based response cache as a final safety net — the demo never goes blank.

4.2 Frontend

- **React 18 + Vite + React Router:** seven-page SPA — Dashboard, Workspace, Proposal Editor, Active Bids, Bid History, Capability Library, and Settings.

- **Live streaming editor:** a custom SSE hook renders the proposal as it is written, token by token.
- **Reusable components:** score gauge, status badges, toast notifications, and a collapsible sidebar.

4.3 Data Layer

- **Supabase (PostgreSQL):** six normalized tables — workspaces, requirements, compliance_items, proposal_sections, bid_scores — with UUID keys and ON DELETE CASCADE integrity.
- **FAISS vector index:** all 50 capability summaries embedded at startup (all-MiniLM-L6-v2 via fastembed) into an in-memory cosine-similarity index.
- **JSONB metadata:** pipeline timings and score breakdowns stored as flexible JSON for analytics.

5. Core Features & Deliverables

Every deliverable in the problem statement is implemented and demonstrable:

Required Deliverable	Status	Implementation
Working prototype accepting a sample RFP	Delivered	Full upload-to-export pipeline, PDF & DOCX
Separate workspace per RFP/RFQ/Tender	Delivered	Isolated workspace with lifecycle status tracking
Auto-generated compliance checklist (pass/fail)	Delivered	Pass / partial / fail per requirement, with confidence and gap notes
Win-probability dashboard	Delivered	Score gauge, six-factor breakdown, sector win rates, model insights
GO/NO-GO decision	Delivered	GO ≥ 70%, CONDITIONAL ≥ 50%, NO-GO < 50%, with top-gap recommendations
≥50% effort reduction (measured)	Exceeded	Per-stage wall-clock timing vs. 6-hour manual baseline; typically >90%
Review / edit / approve UI before export	Delivered	Inline proposal editor with draft / edited / approved statuses

6. AI Components

6.1 Large Language Model — Parsing, Extraction & Generation

Llama 3.3 70B (via GROQ) performs requirement extraction with a rigorously specified JSON schema covering five requirement types: mandatory, evaluation criteria, submission deadline, budget, and question. Documents longer than 14,000 characters are split into overlapping chunks at paragraph boundaries and processed concurrently, then de-duplicated with fuzzy text matching. The same model powers compliance judging (deterministic, temperature 0.0) and

proposal drafting (temperature 0.7), with prompts that cite specific past projects, quantify achievements, and honestly acknowledge gaps.

6.2 Named Entity Recognition — Deterministic Cross-Check

A regex- and dateparser-based NER layer independently extracts deadlines, budget figures (PKR/USD, normalized to millions), evaluation-weight percentages, certifications (ISO 27001/9001/14001, CMMI, PMP, PEC), and clause references. NER findings are injected into the LLM prompt as verified entities, and any deadline or budget the LLM misses is appended automatically. Every requirement is tagged with its extraction source — llm, ner, or both — making the pipeline auditable.

6.3 Hybrid Retrieval-Augmented Generation

Capability matching goes beyond plain vector search. A dense FAISS pass retrieves the top-10 candidates by cosine similarity, which are then re-ranked with structured metadata scoring: domain match (45%), certification match (30%), recency (15%), and client-type fit (10%). The final hybrid score blends dense and structured signals 50/50. An LLM judge then issues a pass / partial / fail verdict per requirement — batched six at a time for efficiency — with a confidence score and a specific gap note when evidence is missing.

6.4 Win-Probability Model & GO/NO-GO Engine

The score blends a trained logistic-regression model (60%) with an explainable weighted heuristic (40%). The model is trained at startup on the 120-bid historical outcomes dataset using compliance rate, score percentage, gap count, budget, and sector win rate as features. The heuristic combines:

Factor	Weight	How It Is Computed
Compliance rate	25%	pass / total (partial counts half)
Domain match	22%	RFP domain vs. capability library domains (alias-aware)
Budget alignment	18%	RFP budget vs. average contract value of matched capabilities
Past win rate	15%	Sector-level win rate from 120 historical bids
Capability depth	10%	Average matching confidence
Competitor presence	10%	Manual input: low / medium / high

7. Technology Stack

Layer	Technology
Frontend	React 18, Vite, React Router v6, lucide-react, custom CSS design system
Backend	Python, FastAPI, Uvicorn (async), Server-Sent Events
Database	Supabase (PostgreSQL), JSONB metadata, cascade-delete integrity
LLM	Llama 3.3 70B via GROQ; OpenRouter fallback; file-based response cache
Embeddings / RAG	fastembed (all-MiniLM-L6-v2, ONNX) + FAISS in-memory index
NER	Regex patterns + dateparser (deterministic, zero-cost)
ML Scoring	scikit-learn LogisticRegression + StandardScaler pipeline
Document I/O	pypdf, python-docx (parsing and professional export)

8. Datasets & Data Pipeline

- **Capability Library (50 records):** past project summaries with domain, certifications, year, contract value, duration, and client type — LLM-enriched and embedded into FAISS at startup.
- **Historical Bid Outcomes (120 rows):** win/loss records with evaluation scores, used to train the win-probability model and compute per-sector win rates (overall historical win rate: 56.7%).
- **Evaluation Criteria Taxonomy (15 entries):** common RFP evaluation criteria mapped onto extracted requirements during parsing.
- **Sample RFPs:** anonymized government/enterprise RFPs (IT services, construction, logistics) used for end-to-end validation.

The Bid History page supports uploading replacement historical datasets, after which sector win rates and model insights refresh automatically.

9. Results & Measured Impact

Every pipeline stage is instrumented with wall-clock timing, persisted per workspace. The dashboard compares total automated processing time against a configurable manual baseline (default: 6 hours per bid, conservative for a 40-80 page RFP).

Metric	Manual Baseline	BidForge AI	Improvement
End-to-end bid preparation	~6 hours	Minutes (typically 2-5)	>90% effort reduction
Requirement extraction	Hours of reading	Automated, with NER cross-check	Auditable source tagging
Compliance mapping	Manual cross-referencing	Automated pass/partial/fail with gap notes	Zero missed clauses in testing
Go/No-Go analysis	Gut feel	Data-driven score with explainable breakdown	Consistent, repeatable

This exceeds the hackathon requirement of a 50% demonstrated reduction in manual bid preparation effort by a wide margin, while keeping a human approver in the loop for quality control.

10. Reliability & Engineering Practices

- **Multi-provider LLM failover:** GROQ → OpenRouter → nearest cached response; the system degrades gracefully rather than failing.

- **Response caching:** every LLM call is cache-first (SHA-256 keyed), cutting cost, latency, and demo risk; cache statistics are visible in Settings.
- **Robust JSON recovery:** multi-stage parsing (direct → markdown strip → regex → line recovery) tolerates noisy model output without crashing.
- **Rate-limit handling:** exponential backoff with bounded concurrency on all LLM batches.
- **Input validation:** file-type and size limits (50 MB), minimum-text guards against scanned PDFs, and a 300k-character cap with explicit truncation warnings.
- **Data integrity:** UUID keys, foreign keys with cascade deletes, and per-workspace temp-file isolation and cleanup.
- **Error surfacing:** specific HTTP status codes and human-readable messages propagated to toast notifications in the UI.

11. Feasibility, Scalability & Roadmap

The backend is stateless — all persistent state lives in PostgreSQL — so it can scale horizontally behind a load balancer. The embedding model is a lightweight ONNX build (~100 MB), keeping deployment cheap. Configuration is fully environment-driven, and the codebase runs on both Windows and Linux.

Planned Enhancements

- Authentication, role-based access, and Supabase row-level security for multi-tenant teams.
- pgvector migration for capability libraries beyond in-memory scale, plus pagination on list endpoints.
- Background job queue (e.g., Celery/Redis) for very large documents and scheduled re-scoring.
- Native PDF export alongside DOCX, and tender-portal submission integrations.
- Continuous model retraining as new bid outcomes are recorded, improving win prediction over time.
- OCR support for scanned tender documents.

12. Conclusion

BidForge AI delivers every deliverable in the problem statement as a working, integrated system: real LLM-powered extraction hardened by deterministic NER, hybrid RAG matching against a genuine capability library, streamed proposal drafting with human review, an explainable trained-model win-probability engine with GO/NO-GO decisions, and professional document export — all organized into per-bid workspaces with measured, instrumented effort savings exceeding 90%.

Beyond the hackathon, the same architecture is a credible foundation for a commercial bid-automation product: the pain point is universal across procurement-heavy industries, the human-in-the-loop design fits real procurement workflows, and the modular AI pipeline allows

each component — extraction, retrieval, scoring — to improve independently as data accumulates.